LABORATORY WORK 6

SQL JOINs

Student: Maman Naskhat
Subject: Database
Topic: SQL JOINs

Objective

To understand and practice different types of SQL JOIN operations, including CROSS JOIN, INNER JOIN, LEFT JOIN, RIGHT JOIN, and FULL JOIN, and to learn the differences between ON and WHERE clauses in join operations.

Part 1: Database Setup

Step 1.1: Create Sample Tables

```
CREATE TABLE employees (
    emp_id INT PRIMARY KEY,
    emp_name VARCHAR(50),
    dept_id INT,
    salary DECIMAL(10, 2)
);

CREATE TABLE departments (
    dept_id INT PRIMARY KEY,
    dept_name VARCHAR(50),
    location VARCHAR(50)
);

CREATE TABLE projects (
    project_id INT PRIMARY KEY,
    project_name VARCHAR(50),
    dept_id INT,
    budget DECIMAL(10, 2)
);
```

Step 1.2: Insert Sample Data

```
INSERT INTO employees (emp_id, emp_name, dept_id, salary)
VALUES
(1, 'John Smith', 101, 50000),
(2, 'Jane Doe', 102, 60000),
(3, 'Mike Johnson', 101, 55000),
(4, 'Sarah Williams', 103, 65000),
(5, 'Tom Brown', NULL, 45000);

INSERT INTO departments (dept_id, dept_name, location)
VALUES
(101, 'IT', 'Building A'),
(102, 'HR', 'Building B'),
(103, 'Finance', 'Building C'),
```

```
(104, 'Marketing', 'Building D');

INSERT INTO projects (project_id, project_name, dept_id, budget)
VALUES
(1, 'Website Redesign', 101, 100000),
(2, 'Employee Training', 102, 50000),
(3, 'Budget Analysis', 103, 75000),
(4, 'Cloud Migration', 101, 150000),
(5, 'AI Research', NULL, 200000);
```

Part 2: CROSS JOIN Exercises

Exercise 2.1: Basic CROSS JOIN

```
SELECT e.emp_name, d.dept_name
FROM employees e
CROSS JOIN departments d;
```

Answer: The result contains 20 rows because there are 5 employees and 4 departments.

$5 \times 4 = 20$ rows.

Exercise 2.2: Alternative CROSS JOIN Syntax

a) Comma notation

```
SELECT e.emp_name, d.dept_name
FROM employees e, departments d;
```

b) INNER JOIN with TRUE condition

```
SELECT e.emp_name, d.dept_name
FROM employees e
INNER JOIN departments d ON TRUE;
```

Exercise 2.3: Practical CROSS JOIN

```
SELECT e.emp_name, p.project_name
FROM employees e
CROSS JOIN projects p;
```

This query displays all possible combinations of employees and projects.

Part 3: INNER JOIN Exercises

Exercise 3.1: Basic INNER JOIN with ON

```
SELECT e.emp_name, d.dept_name, d.location
FROM employees e
INNER JOIN departments d
ON e.dept_id = d.dept_id;
```

Answer: The query returns 4 rows. Tom Brown is not included because his dept_id is NULL, so he does not match any department.

Exercise 3.2: INNER JOIN with USING

```
SELECT emp_name, dept_name, location
FROM employees
INNER JOIN departments USING (dept_id);
```

Answer: USING displays the common dept_id column only once, while ON keeps both table columns available in the result if selected.

Exercise 3.3: NATURAL INNER JOIN

```
SELECT emp_name, dept_name, location
FROM employees
NATURAL INNER JOIN departments;
```

Exercise 3.4: Multi-table INNER JOIN

```
SELECT e.emp_name, d.dept_name, p.project_name
FROM employees e
INNER JOIN departments d
ON e.dept_id = d.dept_id
INNER JOIN projects p
ON d.dept_id = p.dept_id;
```

This query combines employees, departments, and projects using matching department IDs.

Part 4: LEFT JOIN Exercises

Exercise 4.1: Basic LEFT JOIN

```
SELECT
    e.emp_name,
    e.dept_id AS emp_dept,
    d.dept_id AS dept_dept,
    d.dept_name
FROM employees e
LEFT JOIN departments d
ON e.dept_id = d.dept_id;
```

Answer: Tom Brown appears in the result, but his department information is NULL because he is not assigned to any department.

Exercise 4.2: LEFT JOIN with USING

```
SELECT emp_name, dept_id, dept_name
FROM employees
LEFT JOIN departments USING (dept_id);
```

Exercise 4.3: Find Unmatched Records

```
SELECT e.emp_name, e.dept_id
FROM employees e
LEFT JOIN departments d
ON e.dept_id = d.dept_id
WHERE d.dept_id IS NULL;
```

This query finds employees who are not assigned to any existing department.

Exercise 4.4: LEFT JOIN with Aggregation

```
SELECT
    d.dept_name,
    COUNT(e.emp_id) AS employee_count
FROM departments d
LEFT JOIN employees e
ON d.dept_id = e.dept_id
GROUP BY d.dept_id, d.dept_name
ORDER BY employee_count DESC;
```

This query displays all departments and the number of employees in each department, including departments with zero employees.

Part 5: RIGHT JOIN Exercises

Exercise 5.1: Basic RIGHT JOIN

```
SELECT e.emp_name, d.dept_name
FROM employees e
RIGHT JOIN departments d
ON e.dept_id = d.dept_id;
```

This query displays all departments, including departments without employees.

Exercise 5.2: Convert to LEFT JOIN

```
SELECT e.emp_name, d.dept_name
FROM departments d
LEFT JOIN employees e
ON d.dept_id = e.dept_id;
```

Exercise 5.3: Find Departments Without Employees

```
SELECT d.dept_name, d.location
FROM employees e
RIGHT JOIN departments d
ON e.dept_id = d.dept_id
WHERE e.emp_id IS NULL;
```

This query finds departments that do not have any employees.

Part 6: FULL JOIN Exercises

Exercise 6.1: Basic FULL JOIN

```
SELECT
    e.emp_name,
    e.dept_id AS emp_dept,
    d.dept_id AS dept_dept,
    d.dept_name
FROM employees e
FULL JOIN departments d
ON e.dept_id = d.dept_id;
```

Answer:

- NULL values on the left side represent departments without employees.
- NULL values on the right side represent employees without departments.

Exercise 6.2: FULL JOIN with Projects

```
SELECT d.dept_name, p.project_name, p.budget
FROM departments d
FULL JOIN projects p
ON d.dept_id = p.dept_id;
```

Exercise 6.3: Find Orphaned Records

```
SELECT
    CASE
        WHEN e.emp_id IS NULL THEN 'Department without employees'
        WHEN d.dept_id IS NULL THEN 'Employee without department'
        ELSE 'Matched'
    END AS record_status,
    e.emp_name,
    d.dept_name
FROM employees e
FULL JOIN departments d
ON e.dept_id = d.dept_id
WHERE e.emp_id IS NULL OR d.dept_id IS NULL;
```

This query identifies employees without departments and departments without employees.

Part 7: ON vs WHERE Clause

Exercise 7.1: Filtering in ON Clause (Outer Join)

```
SELECT e.emp_name, d.dept_name, e.salary
FROM employees e
LEFT JOIN departments d
ON e.dept_id = d.dept_id
AND d.location = 'Building A';
```

Exercise 7.2: Filtering in WHERE Clause (Outer Join)

```
SELECT e.emp_name, d.dept_name, e.salary
FROM employees e
LEFT JOIN departments d
```

```
ON e.dept_id = d.dept_id
WHERE d.location = 'Building A';
```

Answer:

- Query 1 (ON clause): All employees are included, but only departments in Building A are matched.
- Query 2 (WHERE clause): Only employees whose department is in Building A are returned. Other employees are excluded.

Exercise 7.3: ON vs WHERE with INNER JOIN

Query 1: ON clause

```
SELECT e.emp_name, d.dept_name, e.salary
FROM employees e
INNER JOIN departments d
ON e.dept_id = d.dept_id
AND d.location = 'Building A';
```

Query 2: WHERE clause

```
SELECT e.emp_name, d.dept_name, e.salary
FROM employees e
INNER JOIN departments d
ON e.dept_id = d.dept_id
WHERE d.location = 'Building A';
```

Answer: Both queries return the same results because INNER JOIN only returns matching rows. The filter condition works equivalently in both cases.

Part 8: Complex JOIN Scenarios

Exercise 8.1: Multiple Joins with Different Types

```
SELECT
    d.dept_name,
    e.emp_name,
    e.salary,
    p.project_name,
    p.budget
FROM departments d
LEFT JOIN employees e
ON d.dept_id = e.dept_id
LEFT JOIN projects p
ON d.dept_id = p.dept_id
ORDER BY d.dept_name, e.emp_name;
```

This query displays all departments with their employees and projects, including departments without employees or projects.

Exercise 8.2: Self Join

Add manager_id column

```
ALTER TABLE employees
ADD COLUMN manager_id INT;
```

Update with sample data

```
UPDATE employees SET manager_id = 3 WHERE emp_id = 1;
UPDATE employees SET manager_id = 3 WHERE emp_id = 2;
UPDATE employees SET manager_id = NULL WHERE emp_id = 3;
UPDATE employees SET manager_id = 3 WHERE emp_id = 4;
UPDATE employees SET manager_id = 3 WHERE emp_id = 5;
```

Self JOIN query

```
SELECT
    e.emp_name AS employee,
    m.emp_name AS manager
FROM employees e
LEFT JOIN employees m
ON e.manager_id = m.emp_id;
```

Exercise 8.3: Join with Subquery

Find departments where the average employee salary is above $50,000.

```
SELECT
    d.dept_name,
    s.avg_salary
FROM departments d
INNER JOIN (
    SELECT dept_id, AVG(salary) AS avg_salary
    FROM employees
    GROUP BY dept_id
) s
ON d.dept_id = s.dept_id
WHERE s.avg_salary > 50000;
```

This query uses a subquery to calculate the average salary for each department and displays departments with an average salary above $50,000.

Lab Questions

1. What is the difference between INNER JOIN and LEFT JOIN?

INNER JOIN returns only matching rows from both tables. LEFT JOIN returns all rows from the left table and matching rows from the right table. If there is no match, NULL values are returned.

2. When would you use CROSS JOIN in a practical scenario?

CROSS JOIN is useful when we need all possible combinations of two tables. For example, creating a schedule of all employees and all available projects.

3. Explain why the position of a filter condition (ON vs WHERE) matters for outer joins but not for inner joins.

In OUTER JOIN, ON filters matching rows but keeps unmatched rows. WHERE filters the final result and can remove unmatched rows. In INNER JOIN, both conditions usually produce the same result because only matching rows are returned.

4. What is the result of SELECT COUNT(*) FROM table1 CROSS JOIN table2 if table1 has 5 rows and table2 has 10 rows?

The result is 50 rows.

$5 \times 10 = 50$.

5. How does NATURAL JOIN determine which columns to join on?

NATURAL JOIN automatically finds columns with identical names in both tables and uses them as join conditions.

6. What are the potential risks of using NATURAL JOIN?

NATURAL JOIN can produce unexpected results if tables have additional columns with the same names. It can also become confusing when the database structure changes. Therefore, explicit JOIN conditions are often safer.

7. Convert this LEFT JOIN to a RIGHT JOIN:

Original:

```
SELECT *
FROM A
LEFT JOIN B ON A.id = B.id;
```

Converted:

```
SELECT *
FROM B
RIGHT JOIN A ON A.id = B.id;
```

8. When should you use FULL OUTER JOIN instead of other join types?

FULL OUTER JOIN is useful when we need all records from both tables, including matching and non-matching records. It is especially useful for comparing data and identifying missing records.

Additional Challenges (Optional)

1. Simulate FULL OUTER JOIN using UNION

```
SELECT
    e.emp_name,
    d.dept_name
FROM employees e
LEFT JOIN departments d
ON e.dept_id = d.dept_id

UNION

SELECT
    e.emp_name,
    d.dept_name
FROM employees e
RIGHT JOIN departments d
ON e.dept_id = d.dept_id;
```

2. Find Employees Who Work in Departments That Have More Than One Project

```
SELECT DISTINCT
    e.emp_name,
    d.dept_name
FROM employees e
INNER JOIN departments d
ON e.dept_id = d.dept_id
INNER JOIN projects p
ON d.dept_id = p.dept_id
GROUP BY e.emp_id, e.emp_name, d.dept_name
HAVING COUNT(p.project_id) > 1;
```

3. Show Employee, Manager, and Manager's Manager

```
SELECT
    e.emp_name AS employee,
    m.emp_name AS manager,
    mm.emp_name AS manager_of_manager
FROM employees e
LEFT JOIN employees m
ON e.manager_id = m.emp_id
LEFT JOIN employees mm
ON m.manager_id = mm.emp_id;
```

4. Find All Pairs of Employees Who Work in the Same Department

```
SELECT
    e1.emp_name AS employee1,
    e2.emp_name AS employee2,
    e1.dept_id
FROM employees e1
INNER JOIN employees e2
ON e1.dept_id = e2.dept_id
AND e1.emp_id < e2.emp_id
WHERE e1.dept_id IS NOT NULL;
```

Conclusion

In this laboratory work, I practiced different types of SQL JOIN operations, including CROSS JOIN, INNER JOIN, LEFT JOIN, RIGHT JOIN, and FULL JOIN.

I learned how to combine data from multiple tables, identify unmatched records, use aggregate functions with JOINs, and understand the differences between ON and WHERE clauses.

I also practiced SELF JOIN, subqueries, and more complex SQL queries.

The laboratory work helped me understand how JOIN operations are used to retrieve and analyze related data from relational databases.